This V2 is substantially stronger than V1. If I were reviewing this as a fintech security architect, I'd move it from "good startup design" to "ready for implementation review" with only a handful of remaining changes.

My overall score:

Area	V1	V2
State Machine	B+	A
Fraud Controls	B	A
Auditability	C+	A-
Recovery Design	C	A-
Custodial Readiness	B	A
Partner Agnostic Design	A-	A

Biggest Improvement

The strongest addition is:

Approval Snapshot
+
Pre-Broadcast Verification
+
Reconciling State

This closes the two most common fintech failures:

1. Time-of-check / time-of-use (TOCTOU)
2. Unknown partner outcome

Those are the issues that usually cause loss events.

My Review of the Open Questions

1. Snapshot JSONB vs Separate Immutable Table

Current question:

approval_snapshot JSONB

vs

withdrawal_approval_snapshots

Recommendation

For SchnelPay:

Separate immutable table.

Example:

```
CREATE TABLE withdrawal_approval_snapshots (
    id UUID PRIMARY KEY,
    custodial_transaction_id UUID NOT NULL,
    snapshot JSONB NOT NULL,
    signature VARCHAR(128) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

Reason:

If an attacker modifies:

custodial_transactions

they should not automatically modify:

withdrawal_approval_snapshots

at the same time.

You gain:

* stronger forensic evidence
* better auditability
* separation of duties

For a money-moving system, this is worth the extra table.

Verdict

Separate table wins.

---

2. Halt Worker vs Crash Process

Current proposal:

```
Bad config
↓
Worker stops
↓
Web app continues
```

I strongly agree.

Do NOT crash Express.

Reason:

Users still need:

* login
* support
* audit access
* cancellation ability

You only want:

withdrawal processing

disabled.

Verdict

Keep:

HALT WORKER
NOT WEB PROCESS

———

3. Tamper Detection

Current question:

failed

vs

frozen

Recommendation

Add:

manual_review

or

security_hold

state.

Example:

processing
↓
tamper detected
↓
security_hold

Never:

tamper detected
↓
failed

because failure implies:

normal operational problem

Tampering is:

potential security incident

Those should be distinguishable.

Verdict

Add:

security_hold

———

4. Outbox Now or Later?

Current question:

Build outbox now?

My answer:

Not Yet

You don’t have:

* custody partner
* real withdrawal API
* real idempotency behavior

Without those:

half the outbox design is guesswork.

What you already have:

withdrawal_events
+
reconciling
+
deterministic key

captures most of the value.

Verdict

Build later during partner integration.

Document the future architecture now.

———

5. HMAC Secret Isolation

This is the only place where I think V2 slightly understates the risk.

Current:

APPROVAL_SECRET

stored with application env vars.

This is acceptable at your current stage.

I would document future evolution:

Phase A (now)

Railway Secret

Phase B

Cloud KMS

Examples:

* AWS Key Management Service
* Google Cloud Key Management Service
* Azure Key Vault

Phase C

Partner-side approval controls.

Verdict

Current solution acceptable.

Add roadmap note.

———

6. Event Ledger Immutability

Current:

append-only by convention

I would tighten this.

Recommended

Immediately:

Application-level restriction.

INSERT only

No update route.

No delete route.

Later:

Database trigger.

Example:

```
BEFORE UPDATE
RAISE EXCEPTION
```

and

```
BEFORE DELETE
RAISE EXCEPTION
```

This gives true immutability.

Verdict

Application enforcement now.

DB enforcement before custodial launch.

Additional Threat Missing

I would add one more.

Stale Approval

Scenario:

User requests withdrawal
Admin approves
72 hours pass
Broadcast finally occurs

User account may have changed.

Risk profile may have changed.

Sanctions status may have changed.

Recommendation

Maximum approval age.

Example:

approval older than 24h
↓
requires re-approval

before broadcast.

Especially useful once sanctions screening exists.

Additional Operational Control

Add:

WITHDRAWAL_MAX_DELAY_AGE_HOURS

Example:

24

If:

NOW − approval_time > 24h

then:

processing
↓
manual_review

———

One More Future Requirement

Add to partner checklist:

Withdrawal status lookup by client reference ID

Not merely:

tx hash

because:

tx hash

may not exist yet during reconciliation.

You need:

Did you receive request ABC123?

before blockchain submission.

———

My Recommended V2.1 Changes

I would add the following five items:

1. Separate immutable approval_snapshot table
2. security_hold state
3. Maximum approval age / re-approval rule
4. DB-enforced immutability for withdrawal_events before launch
5. Explicit requirement:
   partner status lookup by client reference ID

After those changes, I would consider the Phase 3.2 architecture mature enough to proceed into implementation review and security testing without major architectural concerns. The remaining risks would be implementation quality, not design quality.